Module No.	Content	Teaching Hours
I	**Introduction:** Introduction to Software Engineering, Software characteristics, Software Crisis, Software Engineering Process. **Software Development Life Cycle (SDLC) Models:** Waterfall, Incremental, Iterative Enhancement, Prototype, RAD and Spiral Models. **Software Requirements Engineering:** Types of Requirements, Requirement Elicitation Techniques Like Interviews, FAST & QFD, Use case Approach, Requirements Analysis Using DFD, Data Dictionaries & ER Diagrams, Requirements Documentation, and SRS. **Software Project Planning:** Size Estimation like Lines of Code & Function Count, Cost. **Estimation Models:** COCOMO (Basic, Intermediate) **Software Design:** Cohesion & Coupling, Classification of Cohesion & Coupling, Function Oriented Design, Object Oriented Design, Structure chart. **Coding:** Characteristics of Coding and Coding style.	20
II	**Software Metrics:** Software Measurements, Token Count, Halstead Software, Measures. **Software Reliability & Quality:** Introduction of Mc Call's & Boehm's Quality Model, Capability Maturity Models **Software Reliability Models**: Basic Execution Time Model. **Software Testing:** **Testing Fundamentals:** Test Case Design, Black Box Testing Strategies, White Box Testing, Unit Testing, Integration Testing, System Testing. **Introduction to Automation Testing and Testing Tools:** Automated Testing Process, Framework for Automation Testing, Introduction to Automation Testing Tool. **Software Maintenance:** Maintenance Process **Maintenance models**: Belady and Lehman Model, Boehm Model Regression Testing, Software Configuration Management; Implementation, Introduction to Reengineering and Reverse Engineering. **Software Risk Management:** Risk Identification and Risk Analysis	20

AUTOMATION TESTING

- Automated testing refers to using specialized software tools to control the execution of tests, compare actual outcomes with predicted outcomes, and generate reports automatically.
- It is an Automation Process of a Manual Process. It allows for executing repetitive tasks without the intervention of a Manual Tester.
- Automation tests can enter test data compare the expected result with the actual result and generate detailed test reports.
- The goal of automation tests is to reduce the number of test cases to be executed manually but not to eliminate manual testing.

WHY TRANSFORM FROM MANUAL TO AUTOMATED TESTING?

- Quality Assurance
- Error or Bug-free Software
- No Human Intervention
- Increased test coverage
- Testing can be done frequently

AUTOMATION TESTING PROCESS

- Test Tool Selection
- Define Scope of Automation
- Planning, Design, and Development
- Test Execution
- Maintenance (Creation of Reports)

FRAMEWORKS FOR AUTOMATION TESTING

- Data-Driven Framework
- Keyword-Driven Framework
- Modular Framework
- Hybrid Framework
- Behaviour-Driven Development (BDD) Framework
- Test-Driven Development (TDD) Framework
- Linear (or Scripted) Framework
- Continuous Integration (CI) Testing Framework…….& many more.

DATA-DRIVEN FRAMEWORK

- A data-driven framework runs test scripts with multiple data sets, stored externally, to ensure the application works with different inputs.
- It enables testing with multiple data sets without duplicating scripts, ideal for validating different inputs and large datasets.
- Example use case: Form submission, login scenarios, input validation.

KEYWORD-DRIVEN FRAMEWORK

- This framework breaks test cases into keywords (e.g., "click", "type", "verify") and data, with keywords listed in a table (often Excel), each linked to a specific action in the script.
- It enables testers with minimal programming skills to write and manage tests by defining keywords and test data, making it more accessible for non-technical users.
- Example use case: Functional testing, UI validations, form input actions.

MODULAR FRAMEWORK

- A modular framework divides the application into smaller components, tests them independently, and creates reusable functions for common actions, which can be used across different test scripts.
- The modular approach ensures reusability, reduces redundancy, and improves maintainability by allowing changes to be made in one place rather than updating all tests.
- Example use case: Testing login, search, and checkout functionality in an e-commerce application, where each module is tested independently.

HYBRID FRAMEWORK

- A hybrid framework combines elements from data-driven, keyword-driven, and modular frameworks etc., offering a flexible and comprehensive testing solution to meet various testing needs.
- Hybrid frameworks offer flexibility by integrating multiple testing methodologies in one framework, making them ideal for handling diverse testing needs in large-scale projects.
- Example use case: Large applications with both data-driven and keyword-driven tests, such as when testing both UI and backend services with different data sets and scenarios.

INTRODUCTION TO AUTOMATION TESTING TOOLS

- **Selenium** (web automation)
- **TestNG** (test framework)
- **Appium** (mobile app automation)
- **Cucumber** (BDD for collaboration)
- **Jenkins** (CI/CD integration)
- **PyTest** (Python testing)

……and many more.

SELENIUM (WEB AUTOMATION)

- **Characteristics:** Open-source tool to automate web browsers, supporting multiple languages (Java, Python, etc.).
- **When to Use:** For automating functional testing of web applications across different browsers.
- **Developed By:** Jason Huggins in 2004; maintained by the Selenium community

TestNG(TEST FRAMEWORK)

- **Characteristics:** Java-based testing framework with support for annotations, parallel execution, and reporting.
- **When to Use:** For unit and integration testing, especially in Java-based projects.
- **Developed By:** Cédric Beust in 2004.

APPIUM (MOBILE APP AUTOMATION)

- **Characteristics:** Open-source tool for automating mobile apps (Android & iOS), supporting native and hybrid apps.
- **When to Use:** For automating functional testing of mobile applications.
- **Developed By:** Jonathan Lipps; maintained by the Appium community.

PYTEST (PYTHON TESTING)

- **Characteristics:** Python-based testing framework supporting unit, integration, and functional testing with fixtures and plugins.
- **When to Use:** For automated testing in Python-based projects.
- **Developed By:** Holger Krekel and maintained by the PyTest community.

JENKINS (CI/CD INTEGRATION)

- **Characteristics:** Open-source automation server for continuous integration (CI) and continuous delivery (CD).
- **When to Use:** For automating build, testing, and deployment pipelines in DevOps environments.
- **Developed By:** Kohsuke Kawaguchi in 2004.